



Universidad Nacional de Lanús

Departamento de Desarrollo Productivo y Tecnológico

**Carrera: Licenciatura en Sistemas**

**Asignatura: INGENIERÍA DE SOFTWARE I**

***Plan 2024 RCS N° 3/23 y 2011 RCS N° 155/11 y  
modificado por N° 179/11***

**Equipo docente:**

***Lic. Romina Mansilla***

***Lic. Leandro Ríos***

***Lic. Solange Lescano***

**Año: 2026**

**Cuatrimestre: 2° Año - 1° Cuatrimestre**

**Carga horaria de la materia: 96**

## ***1- Fundamentación de la Asignatura:***

La rápida expansión de la Informática ha llevado a la escritura de millones de líneas de código antes de que se empezaran a plantear de manera seria metodologías para la construcción y el diseño de sistemas software, así como métodos para resolver los problemas de mantenimiento, fiabilidad, entre otros. Esta expansión sin control tuvo como consecuencia lógica la llamada crisis del software (los productos excedían la estimación de costes, había retrasos en las entregas, las prestaciones no eran las solicitadas, el mantenimiento se hacía extremadamente complicado y a veces imposible, las modificaciones tenían un coste prohibitivo).

La ingeniería del software es una disciplina de la ingeniería cuya meta es el desarrollo costeable de sistemas de software. Éste es abstracto e intangible. No está restringido por materiales, o gobernado por leyes físicas o por procesos de manufactura. De alguna forma, esto simplifica la ingeniería del software ya que no existen limitaciones físicas del potencial del software. Sin embargo, esta falta de restricciones naturales significa que el software puede llegar a ser extremadamente complejo y, por lo tanto, muy difícil de entender.

La intuición va cediendo paso al análisis y los constructores de software comienzan a utilizar medidas cuantitativas de la calidad de los sistemas que construyen para mejorar la forma en que los producen. Los fundamentos matemáticos de la programación se están consolidando merced a la investigación de métodos para la expresión algebraica de los diseños de sistemas software, lo que ayuda a evitar la comisión de errores graves.

La combinación de esfuerzos de las universidades, las empresas y las administraciones públicas orientadas a elevar el desarrollo de software hasta el nivel de una disciplina ingenieril propia de la era industrial se articula en el campo disciplinar llamado Ingeniería del Software.

El área de asignaturas de Ingeniería del Software se orienta a proporcionar a los estudiantes esta concepción ingenieril del proceso de construcción de software mediante la formación en: el Modelado de Sistemas de Información (Ingeniería del Software I), el Control y Gestión de Proyectos de Software (Ingeniería del Software II) y la Calidad y Seguridad de Proyectos de Software (Ingeniería del Software III).

## **2- Objetivos:**

- Introducir al alumno en los conceptos fundamentales de la Ingeniería de Software. En particular profundizar las primeras etapas del ciclo de vida (requerimientos, análisis y diseño de sistemas).
- Adquirir los elementos conceptuales necesarios para especificar, analizar y diseñar software.
- Comprender la importancia de custodiar la calidad y la seguridad en un proyecto de software.

## **3- Contenidos:**

### **UNIDAD 1: CONCEPTOS DE TGS Y SISTEMAS DE INFORMACIÓN**

Conceptos de Teoría General de Sistemas (TGS). Qué es un Sistema. Componentes de un sistema. Conceptos y características (TGS). Métodos de la TGS para el estudio de la realidad. Qué es el pensamiento sistémico. Variedad Interpretativa. Las organizaciones como sistemas. Definición de Sistemas de Información. Objetivos principales. Herramientas a nivel Operativo. Herramientas a nivel Táctico-Estratégico.

#### Bibliografía:

Lorenzon, E. E. (2020). *Sistemas y organizaciones*. EDULP.

<https://doi.org/10.35537/10915/99629>

### **UNIDAD 2: PROCESO Y CICLOS DE VIDA DE SOFTWARE**

Definición de un proceso software. Modelos de Procesos de Software. Aproximación tradicional y ágil. Modelo lineal secuencial. Modelo de construcción de prototipos. Modelo espiral. Scrum. eXtreme Programming. Kanban.

#### Bibliografía:

Bahit, E. (2011-2012). *Scrum y eXtreme programming para programadores*.

<http://umh2818.edu.umh.es/wp-content/uploads/sites/884/2016/02/Scrum-y-eXtreme-Programming-para-programadores.pdf>

- Ochoa, A., Fernández, E., Britos, P., & García-Martínez, R. (2008). *Metodologías de ingeniería informática*. Nueva Librería.
- Pressman, R. S. (2002). *Ingeniería del software: un enfoque práctico* (5.ª ed.). McGraw-Hill.
- Schwaber, K., & Sutherland, J. (2020). *La guía Scrum: la guía definitiva de Scrum: las reglas del juego*. Scrum Alliance. <https://scrumguides.org/docs/scrumguide/v2020/2020-Scrum-Guide-Spanish-European.pdf>
- Van Vliet, H. (2008). *Software engineering: Principles and practice*. John Wiley and Sons.

### **UNIDAD 3: INGENIERÍA DE REQUERIMIENTOS DE SOFTWARE**

El proceso de requerimientos. Tipos de requerimientos. Características de los requerimientos. Notaciones adicionales. Prototipado de requerimientos. Documentación. Participantes en el proceso de especificación de requerimientos. Validación y medición de requerimientos. Selección de técnicas de especificación de requerimientos.

#### **Bibliografía:**

- Pfleeger, S. L., & Atlee, J. M. (2006). *Software engineering: Theory and practice* (3.ª ed.). Prentice Hall.

### **UNIDAD 4: ANÁLISIS Y DISEÑO DE SOFTWARE**

Objetivo del análisis. Modelado conceptual. Técnicas de análisis. Enfoque orientado a procesos, diagrama de flujo de datos: modelo, proceso, secuencia genérica de las actividades a realizar dentro de las metodologías estructuradas. Enfoque orientado a datos, diagrama entidad/relación: modelo, proceso secuencia genérica de las actividades a realizar dentro de las metodologías de la información. Enfoque orientado a objetos: Diagrama de Actividades, Casos de Uso y Diagramas de Casos de Uso, Diagramas de Clases, Diagrama de Secuencia. Enfoque orientado a estados, diagrama de estados: modelo. Diagrama de estados y statechart: proceso, secuencia genérica de las actividades a realizar dentro de las metodologías de tiempo real. Definición de Diseño. Descomposición y modularidad. Estilos arquitectónicos y estrategias. Problemas en la creación del diseño. Características. Técnicas, evaluación y validación del diseño. Documentación de diseño.

### Bibliografía:

Sommerville, I. (2007). *Software engineering* (8.<sup>a</sup> ed.). Pearson.

## **UNIDAD 5: SEGURIDAD DE SOFTWARE**

Tipos de ciclos de vida de desarrollo de software seguro. McGraw's Seven Touchpoints.

Microsoft Security Development Lifecycle (SDL). Correctness by Construction ( CbyC ).

Comprehensive, Lightweight Application Security Process (CLASP). Software Assurance

Maturity Model. Team Software Process (TSP). Modelado de amenazas

### Bibliografía:

Hernández Bejarano, M., & Baquero Rey, L. E. (2020). *Ciclo de vida de desarrollo ágil de software seguro*. Fundación Universitaria Los Libertadores.

## **UNIDAD 6: CALIDAD DE SOFTWARE**

Proceso de Pruebas. Metodología de prueba. Actividades de garantía de calidad del software.

La documentación. Elementos de Configuración. Auditoría.

### Bibliografía:

Boehm, B. W. (1981). *Software engineering economics*. Prentice Hall.

Kan, S. H. (2003). *Metrics and models in software quality engineering* (2.<sup>a</sup> ed.). Addison-Wesley.

Sommerville, I. (2007). *Software engineering* (8.<sup>a</sup> ed.). Pearson.

## ***4- Metodología de Trabajo:***

Por las características de la asignatura, el dictado de la misma contempla un desarrollo teórico-práctico. Aproximadamente se utiliza la mitad del tiempo de cada clase para dictar temas teóricos y la otra mitad para temas prácticos asociados:

- De los temas teóricos se desarrollan los temas del programa exponiendo los conceptos fundamentales de cada uno de ellos así como la propuesta de guías de estudio a fin de propender el debate de los tópicos tratados en clase.

- Para los temas prácticos se realizan ejercicios de ejemplo en clase, a partir de los cuales los estudiantes podrán practicar mediante una guía de ejercicios propuestos y tareas grupales. A su vez, en cada clase, se tratarán las dudas surgidas.

## ***5- Desarrollo de Actividades Prácticas:***

### Objetivos:

- El objetivo de las mismas es afianzar y aplicar los conocimientos adquiridos en las clases teóricas.
- Cada guía de estudio estará conformada por una serie de preguntas y ejercicios que deberán desarrollarse y debatirse.
- En cuanto a los temas prácticos el trabajo de resolución de los ejercicios propuestos en el ámbito de grupos de trabajo permite que el estudiante discuta con pares la validez de sus apropiaciones conceptuales, y genere las primeras ratificaciones o rectificaciones de las mismas. Este proceso de ajuste (ratificación/rectificación) permite al estudiante identificar conceptos cuyo intento de apropiación basada en los saberes logrados hasta ese momento, está fuera de su alcance. Esta situación motiva el proceso de consulta al docente. El enunciado de los ejercicios se basará en la descripción de un problema, debiendo el grupo de estudiantes trabajar en la resolución del mismo a través de las herramientas indicadas para cada caso.

### Metodología:

Se desarrollarán distintas guías de estudio para las unidades con contenidos teóricos, además de una guía de ejercicios perteneciente a la evaluación de las unidades con contenidos prácticos.

- Guía de Estudio 1: Teoría General de Sistemas - Sistemas de Información - Proceso de Software
- Guía de Estudio 2: Ciclos de vida - Tradicionales y Ágiles
- Guía de Estudio 3: Requerimientos de Software
- Guía de Estudio 4: Seguridad de Software
- Guía de Estudio 5: Calidad de Software
- Guía de Ejercicios: Análisis y Diseño - Estructurado y Orientado a Objetos

## **6- Evaluación y Acreditación:**

Durante el desarrollo del curso, la valoración del manejo de conceptos y aplicación de conocimientos se realiza mediante dos evaluaciones parciales. En el diseño de los mismos se utilizarán las siguientes técnicas de evaluación: preguntas de elección múltiple (multiple-choice), sentencias con determinación verdadero/falso, ejercicios de asociación de conceptos y preguntas a desarrollar.

La evaluación del dominio de técnicas prácticas es a través de la resolución de la guía de ejercicios de forma grupal. Se tendrá en cuenta presentación en tiempo y forma (plazo y formato establecido), método de resolución (aplicación del método descrito y correcto uso de las herramientas) y exposición de resultados (coloquio con el equipo docente). En caso de desaprobación, el grupo contará con una fecha de re-entrega.

Para acreditar la regularidad, se requiere un mínimo del 75% de asistencia a las clases. Además, se debe obtener una calificación mínima de 4 (cuatro) puntos en ambos parciales o en sus respectivas instancias de recuperación. Se deberá rendir un examen final que requiere una nota mínima de 4 (cuatro) puntos para su aprobación. La nota definitiva será la obtenida en el examen final.

## **7- Bibliografía:**

Bahit, E. (2011-2012). *Scrum y eXtreme programming para programadores*.

<http://umh2818.edu.umh.es/wp-content/uploads/sites/884/2016/02/Scrum-y-eXtreme-programming-para-programadores.pdf>

Boehm, B. W. (1981). *Software engineering economics*. Prentice Hall.

Hernández Bejarano, M., & Baquero Rey, L. E. (2020). *Ciclo de vida de desarrollo ágil de software seguro*. Fundación Universitaria Los Libertadores.

Kan, S. H. (2003). *Metrics and models in software quality engineering* (2.<sup>a</sup> ed.). Addison-Wesley.

Lorenzon, E. E. (2020). *Sistemas y organizaciones*. EDULP.

<https://doi.org/10.35537/10915/99629>

Ochoa, A., Fernández, E., Britos, P., & García-Martínez, R. (2008). *Metodologías de ingeniería informática*. Nueva Librería.

Pfleeger, S. L., & Atlee, J. M. (2006). *Software engineering: Theory and practice* (3.<sup>a</sup> ed.). Prentice Hall.

- Pressman, R. S. (2002). *Ingeniería del software: un enfoque práctico* (5.<sup>a</sup> ed.). McGraw-Hill.
- Schwaber, K., & Sutherland, J. (2020). *La guía Scrum: la guía definitiva de Scrum: las reglas del juego*. Scrum Alliance. <https://scrumguides.org/docs/scrumguide/v2020/2020-Scrum-Guide-Spanish-European.pdf>
- Sommerville, I. (2007). *Software Engineering* (8.<sup>a</sup> ed.). Pearson.
- Van Vliet, H. (2008). *Software engineering: Principles and practice*. John Wiley and Sons.
- Von Bertalanffy, L. (1976). *Teoría general de los sistemas: fundamentos, desarrollo y aplicaciones*. Fondo de Cultura Económica.